

# Aula 4 — Benchmark Textual entre Modelos

Este notebook compara **Kimi**, **Claude Opus** e **GPT-5.5** em tarefas textuais.

**Objetivo:** medir custo, tempo e qualidade das respostas para o mesmo prompt.

**Métricas:** - Tokens de entrada e saída - Tempo de resposta - Custo estimado - Qualidade da resposta (avaliação manual)

## 1. Instalação das dependências

Descomente e execute a linha abaixo caso ainda não tenha instalado.

```
# !pip install openai anthropic python-dotenv pandas matplotlib
```

## 2. Configuração do ambiente

```
import os
import time
import pandas as pd
import matplotlib.pyplot as plt
from dotenv import load_dotenv

# Clientes das APIs
import openai
import anthropic

load_dotenv()

KIMI_API_KEY = os.getenv("KIMI_API_KEY")
ANTHROPIC_API_KEY = os.getenv("ANTHROPIC_API_KEY")
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")

# Inicialização dos clientes
kimi_client = openai.OpenAI(api_key=KIMI_API_KEY, base_url="https://api.moonshot.ai/v1")
anthropic_client = anthropic.Anthropic(api_key=ANTHROPIC_API_KEY)
openai_client = openai.OpenAI(api_key=OPENAI_API_KEY)
```

## 3. Tabela de preços

Atualize os valores conforme a tabela oficial de cada provider no momento da gravação da aula.

```
# Preços por 1M tokens (USD) – verificados em 2026-08-03.
# As chaves precisam bater com o campo "provider" retornado por cada call_*.
PRICES = {
    "kimi": {"input": 3.00, "output": 15.00}, # kimi-k3 (Moonshot)
    "claude-opus": {"input": 5.00, "output": 25.00}, # claude-opus-5 (Anthropic)
    "gpt-5.5": {"input": 5.00, "output": 30.00}, # gpt-5.5 (OpenAI), faixa <272K
}
```

## 4. Funções padronizadas de chamada

```
def call_kimi(system_prompt, user_prompt, model="kimi-k3"):
    start = time.time()
    response = kimi_client.chat.completions.create(
        model=model,
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_prompt}
        ],
        temperature=1.0
    )
    elapsed = time.time() - start
    return {
        "provider": "kimi",
        "model": model,
        "response": response.choices[0].message.content,
        "input_tokens": response.usage.prompt_tokens,
        "output_tokens": response.usage.completion_tokens,
        "total_tokens": response.usage.total_tokens,
        "time_seconds": elapsed
    }

def call_claude(system_prompt, user_prompt, model="claude-opus-5"):
    start = time.time()
    # O claude-opus-5 não aceita temperature (400 invalid_request_error) e vem com
    # thinking ligado por padrão, que consome parte do max_tokens – por isso o
    # limite é mais folgado que os 4096 usados no opus-4-6.
    response = anthropic_client.messages.create(
        model=model,
        max_tokens=16000,
        system=system_prompt,
        messages=[{"role": "user", "content": user_prompt}]
    )
    elapsed = time.time() - start
    # Com thinking ligado, content[0] é um bloco 'thinking'; o texto vem depois.
    text = next(b.text for b in response.content if b.type == "text")
    return {
        "provider": "claude-opus",
        "model": model,
        "response": text,
        "input_tokens": response.usage.input_tokens,
        "output_tokens": response.usage.output_tokens,
        "total_tokens": response.usage.input_tokens + response.usage.output_tokens,
        "time_seconds": elapsed
    }

def call_openai(system_prompt, user_prompt, model="gpt-5.5"):
    start = time.time()
    # O gpt-5.5 aceita apenas o temperature padrão (1). Enviar outro valor devolve
    # 400 unsupported_value, por isso o parâmetro é omitido aqui.
    response = openai_client.chat.completions.create(
        model=model,
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_prompt}
        ]
    )
    elapsed = time.time() - start
    return {
        "provider": "gpt-5.5",
        "model": model,
        "response": response.choices[0].message.content,
        "input_tokens": response.usage.prompt_tokens,
        "output_tokens": response.usage.completion_tokens,
        "total_tokens": response.usage.total_tokens,
        "time_seconds": elapsed
    }
```

```
}
```

## 5. Funções de cálculo de custo e execução do benchmark

```
def calculate_cost(provider, input_tokens, output_tokens):
    prices = PRICES.get(provider, {})
    input_cost = (input_tokens / 1_000_000) * prices.get("input", 0)
    output_cost = (output_tokens / 1_000_000) * prices.get("output", 0)
    return round(input_cost + output_cost, 6)

# Nome do provider por função de chamada – usado quando a call falha,
# para que a linha de erro use a mesma chave de PRICES/quality_scores.
CALLER_PROVIDERS = {
    "call_kimi": "kimi",
    "call_claude": "claude-opus",
    "call_openai": "gpt-5.5",
}

# Colunas métricas (sem o texto da resposta) – úteis para exibir tabelas e gráficos.
METRIC_COLUMNS = [
    "task", "provider", "model",
    "input_tokens", "output_tokens", "total_tokens",
    "time_seconds", "cost_usd",
]

def run_textual_benchmark(system_prompt, user_prompt, task_name="tarefa"):
    """
    Executa o mesmo prompt nos 4 modelos e retorna um DataFrame comparativo.
    O texto gerado fica na coluna "response" (última do DataFrame).
    """
    results = []
    callers = [call_kimi, call_claude, call_openai]

    for caller in callers:
        try:
            result = caller(system_prompt, user_prompt)
            result["cost_usd"] = calculate_cost(
                result["provider"],
                result["input_tokens"],
                result["output_tokens"]
            )
            result["task"] = task_name
            results.append(result)
        except Exception as e:
            results.append({
                "provider": CALLER_PROVIDERS.get(caller.__name__, caller.__name__),
                "model": "error",
                "response": f"[ERROR] {type(e).__name__}: {e}",
                "input_tokens": 0,
                "output_tokens": 0,
                "total_tokens": 0,
                "time_seconds": 0,
                "cost_usd": 0,
                "task": task_name
            })

    df = pd.DataFrame(results)
    # Mantém "response" no DataFrame – display_responses() depende dela.
    return df[METRIC_COLUMNS + ["response"]]

def metrics(df):
    """Versão só com métricas, para exibir a tabela sem o texto das respostas."""
    return df[METRIC_COLUMNS]

def display_responses(df):
    for _, row in df.iterrows():
```

```
text = str(row["response"])
print(f"\n{'='*60}")
print(f"PROVIDER: {row['provider'].upper()} | MODEL: {row['model']}")
print(f"TOKENS: {row['total_tokens']} | TIME: {row['time_seconds']:.2f}s | COST:
${row['cost_usd']:.6f}")
print(f"{'='*60}")
print(text[:800] + "..." if len(text) > 800 else text)
print()
```

---

## 6. Benchmark 1 — Resumo de texto técnico

```
system_prompt = "Você é um especialista em tecnologia. Resuma o texto de forma clara e objetiva, mantendo os pontos técnicos principais."
```

```
texto_tecnico = """
Large Language Models (LLMs) são modelos de inteligência artificial baseados na arquitetura Transformer, treinados com grandes quantidades de texto para prever a próxima palavra em uma sequência. Eles são capazes de gerar texto, responder perguntas, resumir documentos, traduzir idiomas e auxiliar na programação. Modelos como GPT-4, Claude, Gemini e Kimi utilizam técnicas como attention mechanisms, fine-tuning e reinforcement learning from human feedback (RLHF) para melhorar a qualidade das respostas. A escolha de um LLM para um projeto depende de fatores como custo por token, tamanho de contexto, latência, multimodalidade e capacidade de seguir instruções complexas.
"""
```

```
user_prompt = f"Resuma o seguinte texto em no máximo 3 parágrafos:\n\n{texto_tecnico}"
```

```
df_resumo = run_textual_benchmark(system_prompt, user_prompt, task_name="resumo")
metrics(df_resumo)
```

```
display_responses(df_resumo)
```

---

## 7. Benchmark 2 — Explicação de conceito

```
system_prompt = "Você é um professor de programação. Explique conceitos técnicos de forma simples, com exemplos práticos."
```

```
user_prompt = """Explique o que é uma API REST e por que ela é importante no desenvolvimento web. Use no máximo 200 palavras e inclua um exemplo prático."""
```

```
df_conceito = run_textual_benchmark(system_prompt, user_prompt, task_name="explicacao_conceito")
metrics(df_conceito)
```

```
display_responses(df_conceito)
```

---

## 8. Benchmark 3 — Geração de ideias

```
system_prompt = "Você é um Product Manager experiente. Gere ideias criativas e viáveis para produtos digitais."
```

```
user_prompt = """Sugira 3 funcionalidades para um aplicativo de produtividade que use IA para ajudar desenvolvedores a gerenciar tarefas. Para cada funcionalidade, dê um nome, descreva o problema que resolve e explique como a IA seria usada."""
```

```
df_ideias = run_textual_benchmark(system_prompt, user_prompt, task_name="geracao_ideias")
metrics(df_ideias)
```

```
display_responses(df_ideias)
```

---

## 9. Consolidação dos resultados

```
df_all = pd.concat([df_resumo, df_conceito, df_ideias], ignore_index=True)
metrics(df_all)
```

---

## 10. Visualização comparativa

```
fig, axes = plt.subplots(1, 3, figsize=(16, 4))

# Custo por tarefa
df_all.pivot(index="provider", columns="task", values="cost_usd").plot.bar(
    ax=axes[0], title="Custo por Tarefa e Provider", rot=0
)
axes[0].set_ylabel("Custo (USD)")

# Tempo por tarefa
df_all.pivot(index="provider", columns="task", values="time_seconds").plot.bar(
    ax=axes[1], title="Tempo de Resposta por Tarefa", rot=0
)
axes[1].set_ylabel("Tempo (s)")

# Tokens de saída por tarefa
df_all.pivot(index="provider", columns="task", values="output_tokens").plot.bar(
    ax=axes[2], title="Tokens de Saída por Tarefa", rot=0
)
axes[2].set_ylabel("Tokens de saída")

plt.tight_layout()
plt.show()
```

---

## 11. Avaliação de qualidade

Após analisar as respostas, atribua uma nota de 1 a 5 para cada provider em cada tarefa.

Critérios sugeridos: - Clareza e organização - Fidelidade ao pedido - Profundidade técnica - Concisão

```
# Exemplo de notas manuais (substitua pelas suas avaliações reais)
quality_scores = {
    ("kimi", "resumo"): 4,
    ("claude-opus", "resumo"): 5,
    ("gpt-5.5", "resumo"): 4,
    ("kimi", "explicacao_conceito"): 5,
    ("claude-opus", "explicacao_conceito"): 5,
    ("gpt-5.5", "explicacao_conceito"): 4,
    ("kimi", "geracao_ideias"): 4,
    ("claude-opus", "geracao_ideias"): 5,
    ("gpt-5.5", "geracao_ideias"): 4,
}

df_all["quality_score"] = df_all.apply(
    lambda row: quality_scores.get((row["provider"], row["task"]), 3), axis=1
)

# Custo-benefício: pontos de qualidade por dólar gasto
df_all["efficiency"] = df_all.apply(
    lambda row: round(row["quality_score"] / max(row["cost_usd"], 0.000001), 2), axis=1
)

df_all[["task", "provider", "cost_usd", "quality_score", "efficiency"]]
```

```
# Gráfico de custo-benefício
df_efficiency = df_all.pivot(index="provider", columns="task", values="efficiency")
```

```
df_efficiency.plot.bar(figsize=(10, 5), title="Custo-Benefício: Qualidade por Dólar", rot=0)
plt.ylabel("Pontos de qualidade / USD")
plt.tight_layout()
plt.show()
```

---

## 12. Exportação dos resultados

```
# Métricas (planilha enxuta) + respostas completas em arquivo separado
df_all[METRIC_COLUMNS + ["quality_score", "efficiency"]].to_csv(
    "benchmark_textual_resultados.csv", index=False
)
df_all.to_csv("benchmark_textual_respostas.csv", index=False)
print("Resultados salvos em benchmark_textual_resultados.csv")
print("Respostas completas salvas em benchmark_textual_respostas.csv")
```

---

## 13. Conclusão da aula

Com base nos dados coletados, responda:

1. Qual modelo teve o **menor custo** em cada tarefa?
2. Qual modelo teve a **melhor qualidade** de resposta?
3. Qual modelo teve o **melhor custo-benefício**?
4. Em quais cenários o Kimi se mostrou competitivo em relação aos concorrentes?

Use os dados do DataFrame para justificar suas respostas.